A PROJECT REPORT

ON

ADVANCE JAVA LAB

(SOFTWARE PROJECT)

CAMPUS SHIRE:

COLLEGE PLACEMENT AND INTERNSHIP MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Artificial Intelligence And Data Science

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Yash Kumar (24EARAD182)
Virender Singh (24EARAD179)
Udit Bhardwaj (24EARAD167)
Vaibhav Charan (24EARAD168)
Rahul Tailor (24EARAD130)

Under the Guidance of:

Er. Ram Babu Buri
Assistant Professor, Dept. of Computer Science and Engineering

Session: 2026–27

Department of Artificial Intelligence And Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence And Data Science

CERTIFICATE OF APPROVAL

The Project Report entitled **"Campus shire: College Placement and Internship Management System"** submitted by **"Yash Kumar (24EARAD182), Virender Singh (24EARAD179), Udit Bhardwaj (24EARAD167), Vaibhav Charan (24EARAD168) and Rahul Tailor (24EARAD130)"** has been examined by us and is hereby approved for carrying out the project leading to the award of the degree **"Bachelor of Technology in Artificial Intelligence & Data Science"**. By this approval, the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approves the pursuance of the project only for the above-mentioned purpose.

Project Guide
Er. Ram Babu Buri
Assistant Professor

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence And Data Science

CERTIFICATE

This is to certify that the work embodied in the project entitled **"Campus shire: College Placement and Internship Management System"**, being submitted by **"Yash Kumar (24EARAD182), Virender Singh (24EARAD179), Udit Bhardwaj (24EARAD167), Vaibhav Charan (24EARAD168) and Rahul Tailor (24EARAD130)"** in partial fulfillment of the requirement for the award of **"Bachelor of Technology in Artificial Intelligence And Data Science"** to Rajasthan Technical University, Kota (Raj.) during the academic year 2026–27, is a record of bonafide work carried out by them under our supervision and guidance in the **Department of Artificial Intelligence And Data Science, Arya College of Engineering & Information Technology, Jaipur**.

Project Guide	**HOD**
Er. Ram Babu Buri	**Dr. Ashok Kajla**
Assistant Professor	Professor

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence And Data Science

DECLARATION

We, the students of **Bachelor of Technology in Artificial Intelligence & Data Science**, session **2026–27**, Arya College of Engineering & Information Technology, Jaipur, hereby declare that the work presented in the project entitled **"Campus shire: College Placement and Internship Management System"** is our project work and is correct to the best of our knowledge. Agentic AI was used as a development aid; responsibility for reviewing the work and documenting its limitations remains with the student team.

The work has been carried out with due regard to engineering ethics. Third-party technical material is acknowledged in the references. The report distinguishes supplied implementation information from proposed designs and unexecuted validation plans. This work has not been submitted elsewhere for the award of a degree or professional diploma.

Yash Kumar (24EARAD182)
Virender Singh (24EARAD179)
Udit Bhardwaj (24EARAD167)
Vaibhav Charan (24EARAD168)
Rahul Tailor (24EARAD130)
Date: _____

Department of Artificial Intelligence And Data Science

ACKNOWLEDGEMENT

We express our sincere gratitude to our Project Guide **Er. Ram Babu Buri** for guidance and encouragement in preparing **Campus shire: College Placement and Internship Management System**. We also thank our HOD **Dr. Ashok Kajla** and the Department of Artificial Intelligence And Data Science for providing an academic environment in which we could study web application architecture and recruitment workflows.

This project connects classroom concepts with the practical requirements of students, recruiters and a college placement office. We acknowledge the role of official technical documentation in understanding Java, Servlets, JSP, JDBC, MySQL and deployment practices. Agentic AI supported project development; its use does not replace our responsibility for code review, correctness and truthful reporting.

We thank our parents, friends and classmates for their support and constructive suggestions. Their encouragement helped us organize the work and prepare a project-specific report rather than reuse the unrelated functionality of the reference document.

Finally, we appreciate all who supported the preparation of this report. Any remaining errors or omissions are the responsibility of the project team.

Thanking you

TABLE OF CONTENTS

Section	Page
1. Introduction	**8**
1.1 Overview	8
1.2 Problem Statement	8
1.3 Evidence Boundary	8
1.4 Objectives	9
1.5 Scope and Applications	9
1.6 Out-of-Scope Claims	9
1.7 Stakeholder Responsibilities	10
1.8 Functional Requirements	10
1.9 Non-Functional Requirements	10
2. Software & technology architecture	**11**
2.1 Technologies Identified by the Team	11
2.2 Compatibility Considerations	11
2.3 Supplied Directory Organization	12
2.4 Configuration Responsibilities	12
2.5 Reproducible Build Checklist	12
2.6 Student Identification	13
2.7 Module Ownership Record	13
2.8 Shared Accountability	13
3. Design & methodology	**14**
3.1 MVC-Style System Architecture	14
3.2 Request–Response Methodology	14
3.3 Sessions and AuthFilter	15
3.4 Proposed Access Matrix	15
3.5 Object-Level Checks	15
3.6 Session Safety	15
3.7 Company Registration and Maintenance	16
3.8 Jobs and Internship Postings	16
3.9 Applicant Review	16
3.10 Student Requirements	17
3.11 TPO / Administrator Functions	17
3.12 Operational Review	17
3.13 Reporting Semantics	17
3.14 Proposed State Model	18
3.15 Proposed Logical Schema	19
3.16 Data Integrity Rules	19
3.17 Audit and Retention	19
3.18 Proposed Entity Relationship Diagram	20

TABLE OF CONTENTS (CONTINUED)

Section	Page
3.19 DAO Responsibilities	20
3.20 Parameterized Operations	20
3.21 Reported Use of AI	21
3.22 Recommended Development Workflow	21
3.23 Evidence and Accountability	21
3.24 Workflows versus Agents	21
3.25 Unverified Runtime Feature Details	22
3.26 Proposed Controlled Agent Boundary	22
3.27 Human Decision Authority	22
3.28 Recommended Threat Review	23
3.29 Failure Responses	23
3.30 Session and Permission Review	23
4. Result and discussion	**24**
4.1 Verification Status	24
4.2 Functional Test Plan	24
4.3 Test Record Requirements	24
4.4 Boundary and Integration Tests	25
4.5 Performance Measurement Plan	25
4.6 Interpretation of Results	25
4.7 Screen Evidence	26
4.8 Recommended Screenshot Set	26
4.9 Screenshot Preparation	26
Application Screenshots (Figures 4.2 to 4.6)	27
4.10 Current Documentation Limitations	28
4.11 Application Review Checklist	28
4.12 Completion Criteria	28
5. Conclusion and recommendation	**29**
5.1 Conclusion	29
5.2 Documented Outcomes	29
5.3 Recommendations for Future Work	29
6. Appendices	**30**
Appendix A: Proposed System Diagrams	30
Appendix B: Application Submission Sequence	31
Appendix C: Illustrative Java DAO Example	32
Appendix D: Illustrative SQL	33
Appendix E: Final Evidence Checklist	33
References	**34**

1. INTRODUCTION

1.1 Overview

Campus shire is a college placement and internship management project intended to organize the interaction between students, companies and the Training and Placement Office (TPO). Its purpose is to maintain recruitment information in a common application instead of scattering it across separate forms, messages and spreadsheets.

According to the supplied project description, the application uses Java for application logic, Servlets for request handling, JSP pages for dynamic views and DAO classes with JDBC for database operations. MySQL stores relational data. Sessions and an AuthFilter support login state and role-based access. These are supplied architectural details, not findings from an independent execution of the application.

1.2 Problem Statement

The project addresses a coordination problem: opportunity details, company information and application decisions need consistent ownership and visibility. A recruitment system should prevent students from losing track of applications, help recruiters review the correct applicants, and allow the TPO to oversee recruitment activity.

The central problem is therefore to provide a structured workflow for publishing jobs and internships, receiving applications and recording decisions while controlling access to personal information. A usable interface is important, but reliable validation and authorization must also be enforced on the server.

1.3 Evidence Boundary

The available inputs are the project title, team details, technology stack, a partial directory tree and descriptions of recruiter and administrator functions. No source archive, database dump, running instance, application screenshots or test logs were supplied. Completion claims in this report are consequently limited to those inputs.

1. INTRODUCTION (CONTINUED)

1.4 Objectives

• Organize company registrations, jobs, internships and applications within one recruitment workflow.

• Provide distinct student, recruiter and TPO access boundaries appropriate to their responsibilities.

• Separate JSP presentation, Servlet request processing and DAO-based persistence.

• Keep application-status changes traceable and avoid duplicate or inconsistent records.

• Use agentic AI responsibly in development and document the limits of any runtime AI assistance.

• Prepare a repeatable test plan covering authorization, data integrity and common failure cases.

1.5 Scope and Applications

The intended scope is institutional recruitment administration, not an open marketplace that independently verifies every employer. The system can support campus placement drives, internship opportunities and applicant review. Its usefulness depends on the accuracy of company records, the clarity of posting requirements and the reliability of status updates.

Student profile management, opportunity browsing and application submission are discussed as expected domain requirements. Their exact screens and completion state must be confirmed from the application. Placement statistics, automated notifications and AI recommendation tools must not be described as operational unless supporting implementation evidence exists.

1.6 Out-of-Scope Claims

This report does not claim measured placement improvement, a deployment at the college, verified AI matching accuracy or completed load testing. It also does not guarantee hiring outcomes. The college and employer remain responsible for recruitment policy and final decisions.

1. REQUIREMENTS AND STAKEHOLDERS

1.7 Stakeholder Responsibilities

Stakeholder	Expected responsibility
Student	Maintain relevant information, evaluate opportunities and follow application updates; exact module requires inspection.
Company / recruiter	Maintain company details, publish jobs and internships, review applicants and update statuses as described by the team.
TPO / administrator	Manage recruitment records, review registrations and activity, and inspect reports where implemented.
Project team	Develop, review and document the software, including AI-assisted changes and limitations.

1.8 Functional Requirements

The supplied description identifies company registration and maintenance, opportunity posting, applicant review and administrative management. Each function needs a defined actor, permitted operation, validation rule and observable result. For example, posting an internship should identify its company owner and specify whether it is open for applications.

1.9 Non-Functional Requirements

Recommended requirements include readable forms, understandable error messages, consistent navigation, controlled access and recoverable persistence failures. These are acceptance criteria rather than measured achievements. Sensitive student information should be shown only when needed for an authorized recruitment task.

Requirements should be converted into traceable checks before submission. A requirement marked “complete” should point to a route, view, DAO operation and test record. A feature described only in documentation should remain “specified” until that evidence is available.

2. SOFTWARE & TECHNOLOGY ARCHITECTURE

2.1 Technologies Identified by the Team

Component	Technology	Role
Application logic	Java	Model objects, validation and server-side behavior.
Web tier	Servlets and JSP	Request handling and dynamic web views.
Data access	JDBC and DAO	Database connection and persistence operations.
Database	MySQL	Relational storage of recruitment data.
Frontend	HTML, CSS, JavaScript	Forms, navigation and browser interaction.
Container	Apache Tomcat	Hosts the Java web application.
Build	Maven	Project and dependency configuration.
Version control	Git and GitHub	Source history and repository hosting.

2.2 Compatibility Considerations

The installed Java version, Tomcat version, dependency versions and Servlet API namespace were not supplied. They should be read from the actual build configuration and runtime before deployment. Do not infer a specific release merely from the directory layout.

Tomcat documentation describes Servlet/JSP web application organization and deployment. The source imports and declared dependencies should determine which container family is appropriate; mixing incompatible API namespaces is a deployment risk. See reference [1].

The report keeps the Advance Java Lab title from the blueprint while replacing its anonymous-issue-reporting content with the Campus shire domain.

2. PROJECT STRUCTURE AND BUILD

2.3 Supplied Directory Organization

```
campushire/
|-- pom.xml
|-- build.bat
|-- build.ps1
|-- start-tomcat.bat
|-- stop-tomcat.bat
|-- database/
|   `-- placement_system.sql
`-- src/main/
    |-- java/com/placement/
    |   |-- model/
    |   |-- dao/
    |   |-- util/
    |   |-- filter/
    |   `-- servlet/
    |-- resources/db.properties
    `-- webapp/
        |-- WEB-INF/views/
        |-- css/
        |-- js/
        |-- includes/
        |-- index.jsp
        `-- login.jsp
```

The submitted tree ended with an incomplete “reg…” entry. No filename has been invented to complete that entry. The directory name “campushire” is preserved as supplied; the displayed project title remains “Campus shire”.

2.4 Configuration Responsibilities

The team should inspect pom.xml for packaging, dependencies and compiler settings; inspect the SQL script for the actual schema; and inspect db.properties for the configuration keys expected by the connection utility. Build and start scripts must be reviewed before their commands are documented as reliable.

2.5 Reproducible Build Checklist

Record the repository revision, required JDK, compatible Tomcat installation and database initialization procedure. Keep credentials outside public source history. Confirm that the produced artifact deploys and that the login page is reachable. None of these steps has been executed against the project in preparing this report.

2. TEAM STRUCTURE AND RESPONSIBILITIES

2.6 Student Identification

Student name	RTU roll number
Yash Kumar	24EARAD182
Virender Singh	24EARAD179
Udit Bhardwaj	24EARAD167
Vaibhav Charan	24EARAD168
Rahul Tailor	24EARAD130

Only the roll numbers supplied by the team are included. Additional enrollment identifiers have not been inferred from the reference report.

2.7 Module Ownership Record

Individual module assignments were not supplied. A defensible contribution record should list the responsible member, reviewed commits, affected files and validation evidence. The five students above should confirm ownership before adding a contribution table to the final submission.

Work area	Evidence to record
Servlets and authentication	Routes, filter mappings and reviewed login/logout behavior.
Database and DAO	Schema changes, queries, constraints and transaction tests.
JSP and frontend	Views, form validation, navigation and accessibility review.
Recruitment workflow	Posting rules, ownership checks and status transitions.
AI, testing and report	Tool usage notes, test logs and documentation review.

2.8 Shared Accountability

AI-generated changes should be reviewed by the team in the same way as manually written changes. Authorship assistance does not transfer accountability for security, requirements or factual claims to the AI tool.

3. DESIGN & METHODOLOGY

3.1 MVC-Style System Architecture

The supplied architecture places Servlets and Filters between the browser and the model/DAO layer. JSP pages render the views, DAO classes encapsulate data access, JDBC connects Java code to MySQL, and authentication sessions provide request context.

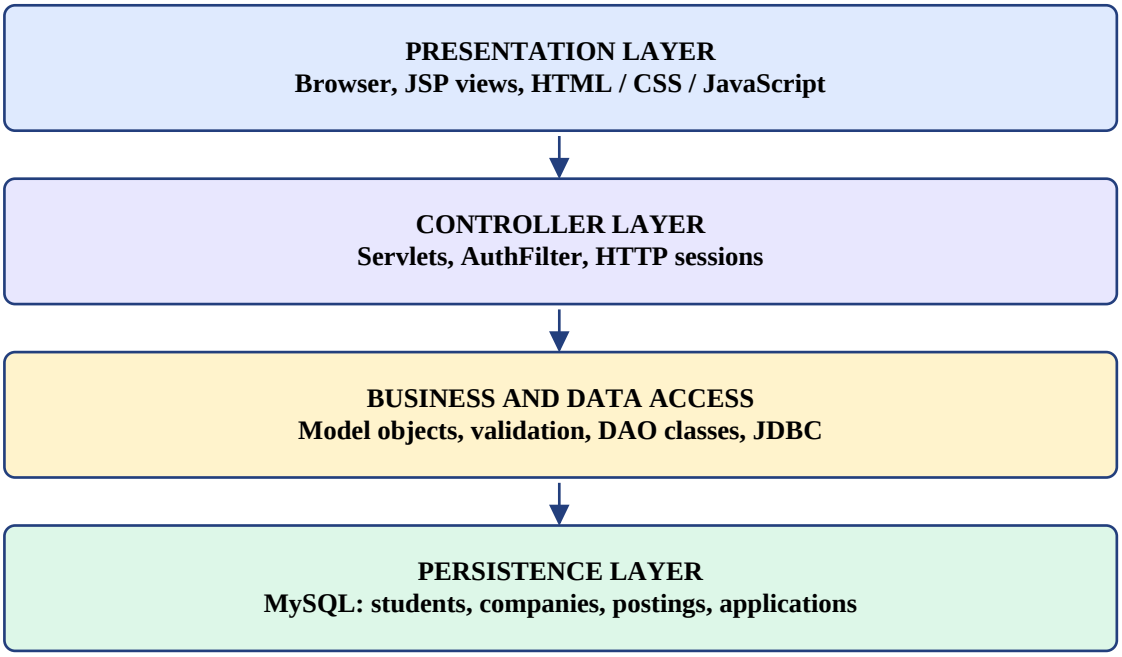


Figure 3.1: Supplied layered architecture

3.2 Request–Response Methodology

A recommended request path is: receive the HTTP request, establish the authenticated actor, validate inputs, apply business and ownership rules, call a DAO method, and return a response or redirect. JSP views should receive prepared view data rather than make their own database queries.

The diagram represents the structure described by the team. It does not demonstrate that all routes follow the same separation or that a distinct service layer already exists. Those questions require source inspection.

3. AUTHENTICATION AND AUTHORIZATION

3.3 Sessions and AuthFilter

The team identifies sessions and AuthFilter as the mechanisms for maintaining login state and role-based access. A session identifies the authenticated actor across requests. The filter should be mapped to protected routes so that access checks occur before protected processing.

3.4 Proposed Access Matrix

Operation	Student	Recruiter	TPO
Edit student information	Own record	No	Authorized scope
Publish opportunity	No	Own company	Policy-defined
Review applicants	Own status only	Own posting	Authorized scope
Manage companies	No	Own details	Authorized scope
Placement reports	Not assumed	Not assumed	Where supported

3.5 Object-Level Checks

Role checks alone are insufficient: a recruiter must not review another company's private applicants by changing an identifier in a URL. Each protected record operation should also verify ownership or an explicit administrative permission. This is a recommended design control, not a verified property of the submitted code. See reference [4].

3.6 Session Safety

Recommended checks include replacing the session identifier after successful login, invalidating sessions at logout, and using appropriate cookie protections and idle limits. Verify these controls in the actual container configuration. See reference [3].

3. COMPANY AND RECRUITER MODULE

3.7 Company Registration and Maintenance

The supplied functional description states that companies or recruiters can register and maintain company details. The precise registration fields, verification policy and account-activation states have not been provided. The report therefore treats these as described functions whose implementation must be inspected.

Recommended company data includes a company name, authorized contact, contact email and a description of recruitment activity. Fields should be checked for format, required values and length. The TPO should define whether company registrations require approval before opportunities become visible.

3.8 Jobs and Internship Postings

Recruiters are described as able to post jobs and internships. A useful posting distinguishes the opportunity type, role title, description, requirements, application deadline and company owner. Salary or stipend fields should not be invented when the employer has not supplied them.

3.9 Applicant Review

The described recruiter workflow includes reviewing applicants and updating application statuses. Every update should identify the authorized reviewer, the application and the permitted next status. Student-visible status labels should be clear enough to distinguish review from an offer.

Validation scenario	Recommended behavior
Missing required posting data	Reject with field-specific errors.
Closed posting	Prevent new applications according to policy.
Different company owner	Deny access to private applicant records.
Unsupported status value	Reject rather than store arbitrary text.

All rules in the table are proposed acceptance criteria; no execution evidence was supplied.

3. STUDENT AND ADMINISTRATOR WORKFLOWS

3.10 Student Requirements

A student-facing recruitment workflow ordinarily needs a profile, an opportunity list and an application view. These are domain requirements used to explain the design, not a claim that the current application implements every screen. The team should verify the available student routes before labeling the module complete.

Recommended profile fields should be limited to information genuinely needed for recruitment. A student should understand which information is visible to a recruiter when an application is submitted. Profile corrections and application withdrawal should follow explicit institutional policy.

3.11 TPO / Administrator Functions

The supplied description includes administrative pages for managing students, companies, postings and applications. It also includes review of company registrations and recruitment activity. Placement-related reports are conditional: they are available only where supported by the actual application.

3.12 Operational Review

Area	What the TPO should verify
Company record	Authorized contact and registration review policy.
Posting	Opportunity details, company ownership and deadlines.
Application	Visible status, correct posting link and valid actor.
Report	Defined metric, reporting period and source records.
Administrative change	Reason, permission and traceable audit event.

3.13 Reporting Semantics

A count of applications is not a count of placed students. A count of offers is not a count of accepted offers. Reports should state their unit of analysis and avoid double-counting a student who applied to several opportunities.

3. APPLICATION LIFECYCLE

3.14 Proposed State Model

The application's actual status constants were not supplied. The following lifecycle is a proposed model for explaining applicant review and must be reconciled with the implemented values. It is not copied from the blueprint's grievance state machine.

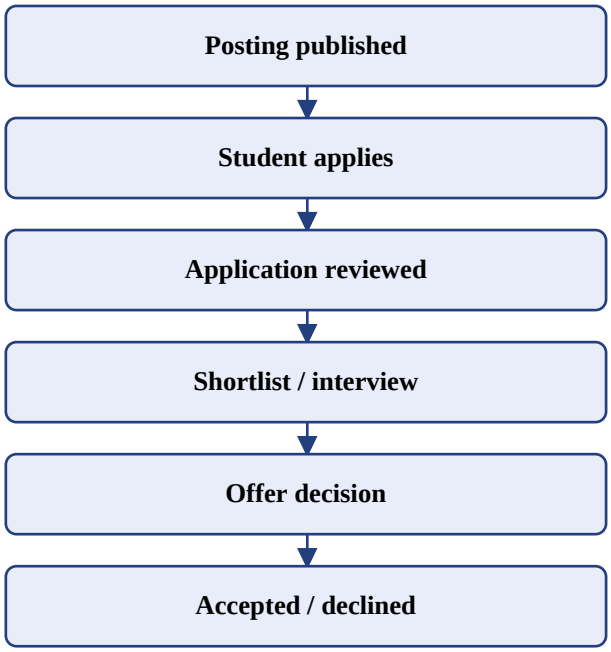


Figure 3.2: Proposed recruitment progression

State / event	Responsible actor and control
Applied → Under review	Authorized reviewer opens the application.
Under review → Shortlisted / Rejected	Recruiter records a screening decision.
Shortlisted → Interview / Rejected	Recruiter records the next selection stage.
Interview → Offered / Rejected	Authorized recruitment decision.
Offered → Accepted / Declined	Student response under institution policy.

Withdrawal, cancellation and posting closure should be handled explicitly. Final status changes must not overwrite the history silently. The diagram shows a main path only; rejected and withdrawn branches require separate policy decisions.

3. DATABASE DESIGN

3.15 Proposed Logical Schema

MySQL is confirmed in the supplied stack, but the contents of the SQL initialization script were not provided. The entities below are a proposed recruitment schema. Names and fields should be mapped to the actual script before claiming schema equivalence.

Entity	Illustrative fields
users	user_id (PK), email, password_hash, role, active
students	student_id (PK), user_id (FK), name, roll_no
companies	company_id (PK), user_id (FK), company_name
postings	posting_id (PK), company_id (FK), type, title, deadline
applications	application_id (PK), student_id (FK), posting_id (FK), status
application_history	history_id (PK), application_id (FK), actor_id, old/new status, timestamp

3.16 Data Integrity Rules

Recommended constraints include unique user email, unique student roll number and a unique student/posting pair for applications. Foreign keys should maintain links among companies, postings and applications. These checks should be implemented in persistence as well as validated in the request path.

3.17 Audit and Retention

A status history can explain who changed an application and when. It should not unnecessarily copy full résumé contents or credentials. Deletion, archival and retention policy should be defined by the institution, especially for records used in historical placement reports.

The proposed schema allows different recruitment records to be linked without reusing the anonymous voting tables from the source report. No original issue, vote or class-verification entity is part of this design.

3. ENTITY RELATIONSHIPS AND DAO DESIGN

3.18 Proposed Entity Relationship Diagram

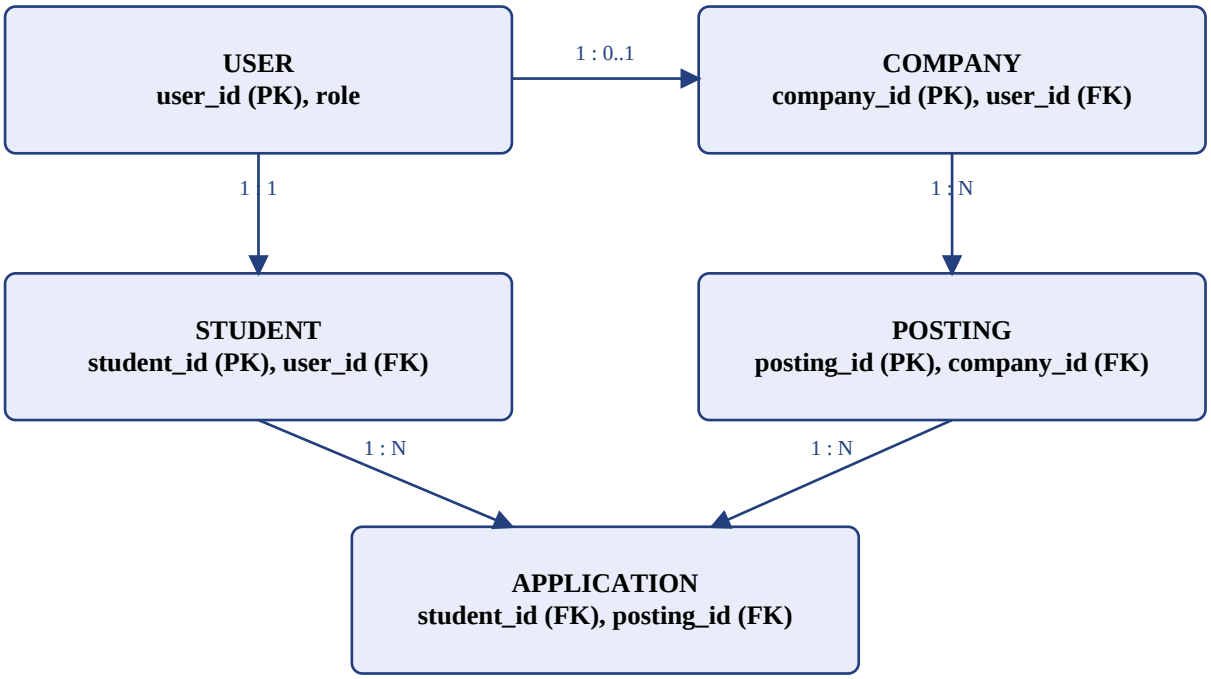


Figure 3.3: Proposed core recruitment relationships

A user can be associated with a student profile or a company account according to role. A company can own multiple postings. A student can submit multiple applications, and a posting can receive multiple applications. The application entity resolves this many-to-many relationship.

3.19 DAO Responsibilities

DAO methods should encapsulate query execution and conversion of database rows into Java model objects. They should accept explicit inputs and return defined outcomes rather than manipulate JSP output. SQL exceptions should be translated into a controlled application response.

3.20 Parameterized Operations

JDBC PreparedStatement supports binding parameter values instead of inserting raw input into SQL text. It is recommended for applicant lookup and record updates. See reference [2]. Queries still need authorization checks and validation; parameter binding alone does not decide who may access a record.

Where a status update and its history entry must succeed together, use a transaction boundary and roll back both if either operation fails. The actual connection utility, pooling strategy and transaction conventions remain unverified.

3. AGENTIC AI IN DEVELOPMENT

3.21 Reported Use of AI

The team states that the project was made using agentic AI and confirmed that AI was used both during development and within the application. The specific development tool, model, framework, agent configuration and runtime AI feature list have not been supplied. This report acknowledges the stated use without inventing those details.

3.22 Recommended Development Workflow

Stage	Suggested AI assistance	Human responsibility
Requirements	Draft tasks from feature descriptions.	Approve scope and acceptance criteria.
Implementation	Suggest code and explain integration.	Review compatibility and security.
Debugging	Analyze supplied errors and propose fixes.	Reproduce and check the fix.
Testing	Generate boundary and negative cases.	Execute and preserve results.
Documentation	Organize architecture and usage notes.	Remove unsupported claims.

3.23 Evidence and Accountability

For academic reproducibility, record the tool name, usage date, task description, reviewed changes and any remaining uncertainty. Do not include secret prompts containing database credentials or student personal data. A suggestion produced by an agent is not a test result.

3.24 Workflows versus Agents

Technical guidance distinguishes predetermined workflows from agents that direct their own tool use. This distinction helps describe the system precisely instead of labeling every automated action agentic. See reference [5]. The team should identify which pattern its application actually uses.

3. RUNTIME AI: DESIGN AND SAFEGUARDS

3.25 Unverified Runtime Feature Details

The team's confirmation of runtime AI does not identify a specific implemented capability. The following examples are design options, not completed Campus shire features: explaining posting requirements, helping a student identify missing profile information, or summarizing recruitment activity for an authorized TPO user.

3.26 Proposed Controlled Agent Boundary

An AI component should receive only the task context it needs and access application data through narrow, authorized operations. The same identity and record-access controls used for ordinary requests should apply to agent tool calls. The model should not receive unrestricted database credentials.

Proposed safeguard	**Purpose**
Read-only default	Prevent accidental changes during assistance.
Explicit approval for writes	Keep decisions and submissions under user control.
Limited task context	Avoid unnecessary exposure of student information.
Timeout and request budget	Bound delays and repeated tool calls.
Non-AI fallback	Keep core recruitment operations available.
Action log	Make tool use and approval traceable.

3.27 Human Decision Authority

Eligibility, application status and hiring decisions should be governed by defined policy, not an unexplained model response. AI output should not promise selection, fabricate employer details or silently submit an application. These are design recommendations and must be validated against the actual integration.

Before including an AI screen in the final project evidence, record its input, tool permissions, model configuration, output and fallback behavior.

3. SECURITY AND ERROR HANDLING

3.28 Recommended Threat Review

The major design concerns are unauthorized record access, malicious form inputs, misuse of recruiter permissions, unsafe file handling and disclosure of configuration secrets. The review should cover direct URL access as well as navigation through the normal interface.

Risk	Recommended control to inspect
Cross-company access	Check company ownership on every applicant operation.
Injection through form fields	Use parameterized SQL and validate values.
Unsafe state-changing request	Require appropriate request method and CSRF protection.
Script content in displayed data	Escape user-controlled values in rendered views.
Résumé upload abuse	If uploads exist, validate type/size and control storage.
Credential exposure	Keep secrets out of public Git history and client output.

3.29 Failure Responses

Database failure should produce a controlled message rather than a stack trace containing connection details. A failed save should not display a success confirmation. Server logs should contain diagnostic context without recording passwords or full sensitive documents.

3.30 Session and Permission Review

Inspect login, logout and expired-session behavior. Test role changes and direct access to administrative routes. Verify record-level permission checks after the filter allows a request. References [3] and [4] inform these recommendations; none is reported as a completed security audit. Security testing should be performed only in an authorized development environment using synthetic records.

4. RESULT AND DISCUSSION

4.1 Verification Status

A running application and source archive were not available for inspection. The supplied descriptions establish the intended stack and described workflows, but not their execution outcome. This chapter therefore presents a validation plan and evidence requirements instead of copying "PASSED" claims from the reference report.

4.2 Functional Test Plan

Test	Expected behavior	Status
Login / logout	Valid login; protected routes denied after logout.	Not executed
Company registration	Required fields checked; record handled by policy.	Not executed
Post opportunity	Authorized recruiter creates valid posting.	Not executed
Submit application	Valid student/posting link; duplicate rejected.	Not executed
Review applicants	Only authorized company or TPO scope shown.	Not executed
Change status	Allowed value and transition stored correctly.	Not executed
Reports	Metric matches eligible source records.	Not executed

4.3 Test Record Requirements

Each executed test should include a case identifier, environment, source revision, synthetic input, expected output, actual output and evidence. A screenshot alone may demonstrate a view, but it does not establish persistence correctness or authorization.

If the current application lacks a feature in this table, mark that case "not implemented" or "out of scope" rather than "failed" or "passed".

4. QUALITY AND PERFORMANCE EVALUATION

4.4 Boundary and Integration Tests

Scenario	Expected outcome / evidence
Repeat application request	At most one record for the same student and posting.
Altered company identifier	Access denied outside the recruiter's ownership.
Deadline boundary	Behavior matches the defined timezone and cutoff policy.
Database disconnect	Controlled error; no false success message.
Concurrent status updates	Conflict handled without silent history loss.
Invalid or missing session	Protected operation denied.
Unsupported report feature	No claim of availability without implementation.

4.5 Performance Measurement Plan

Measure response time with a stated workload and environment, not an assumed number. Record Java/Tomcat configuration, database size, concurrent users and request types. Report median and high-percentile latency alongside the error rate and observation period.

A useful workload should distinguish browsing from writes and recruiter applicant review from TPO reporting. Caching or indexes should be evaluated against real query patterns rather than claimed as implemented from the presence of MySQL.

4.6 Interpretation of Results

No numerical benchmark, test pass rate, security rating or AI accuracy measurement is available for this report. Once tests are executed, results should be tied to the specific build under test. Successful local operation should not be presented as proof of production reliability.

The primary verified report output is a project-specific design and validation document based on the team's supplied information, not a certification that the application is complete.

4. INTERFACE DOCUMENTATION

4.7 Screen Evidence

No application screenshots were supplied. The illustration below is a schematic of a possible student workspace, not a screenshot or a reconstruction of the current interface. Actual captured screens should replace it only after the application is inspected.

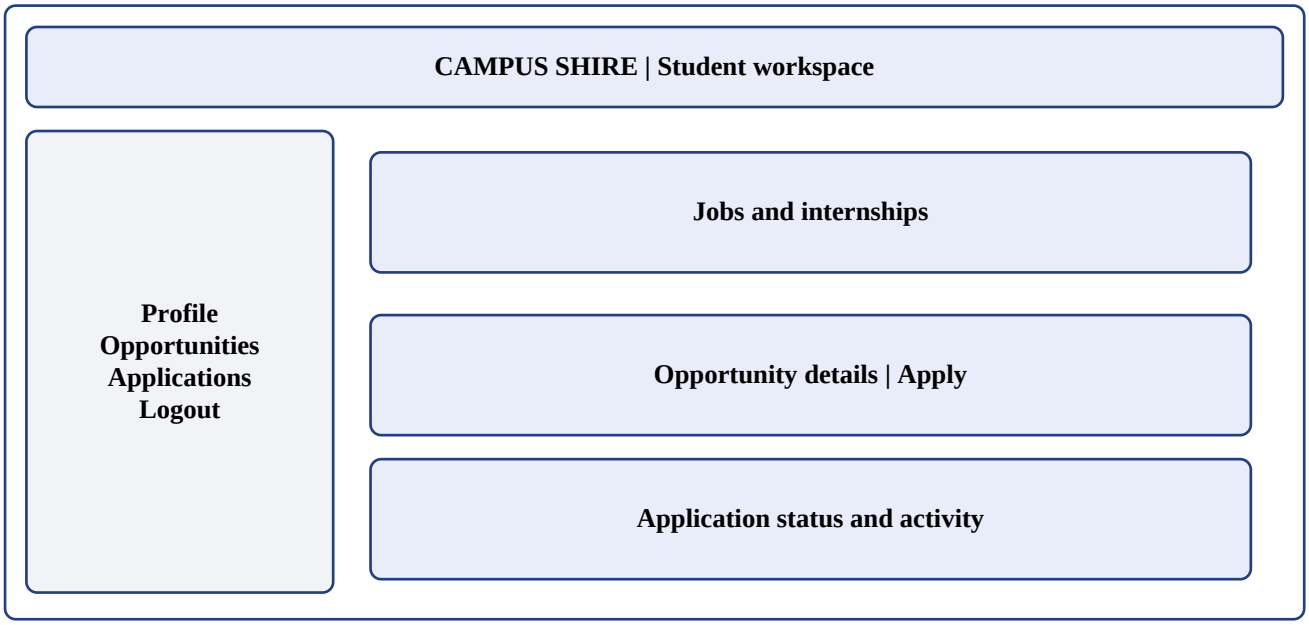


Figure 4.1: Illustrative interface schematic — not an application screenshot

4.8 Recommended Screenshot Set

- Login view showing normal and invalid-login behavior, without exposing credentials.
- Company opportunity form and recruiter applicant-review view, if implemented.
- Student opportunity or application view, if implemented.
- TPO management page and a report view only where supported.

4.9 Screenshot Preparation

Use synthetic student and company records. Caption each screen with the module, user role and build revision. Remove real email addresses, confidential applicant details and session identifiers. Do not reuse the original report's placeholders as evidence of Campus shire functionality.

4. INTERFACE DOCUMENTATION (CONTINUED)

Application Screenshots

Figures 4.2 to 4.6 are screenshots of the application's public pages as supplied by the project team: the landing page, the placement-drive listing and the sign-in page for the Admin / TPO and Student roles. They show how the interface appeared at capture time.

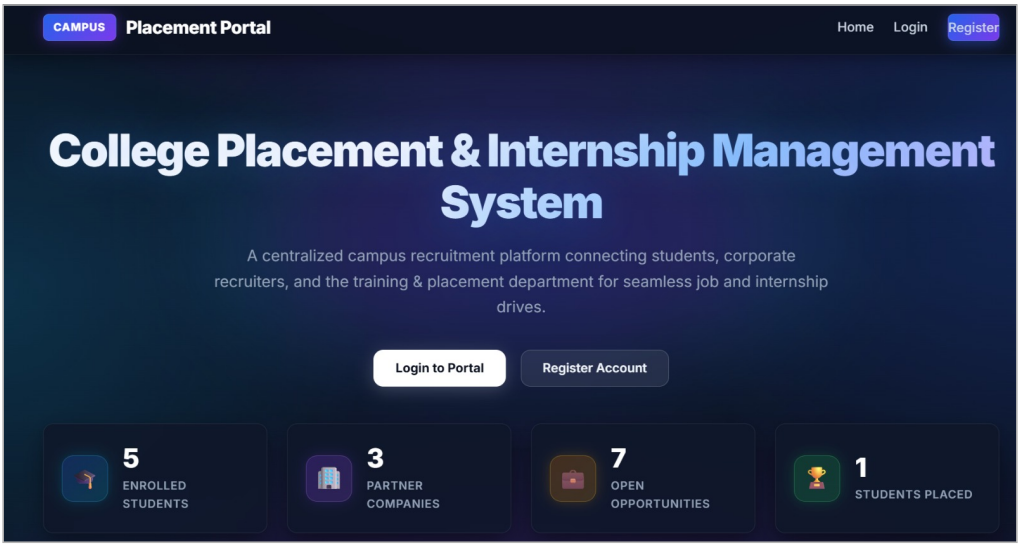


Figure 4.2: Landing page with portal summary counts

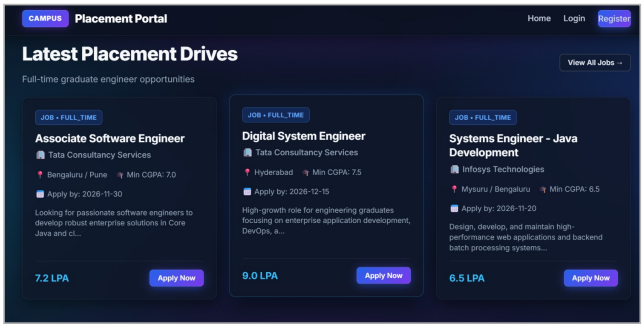


Figure 4.3: Latest placement drives (job cards)

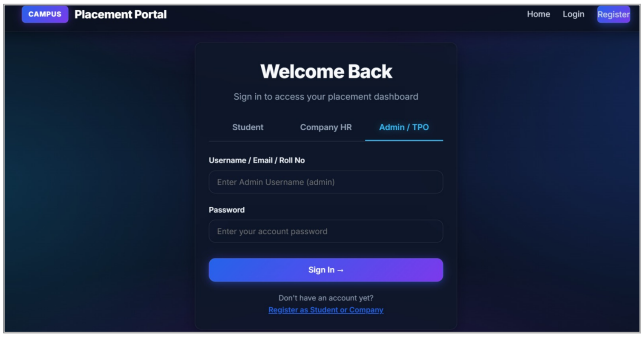


Figure 4.4: Sign-in page, Admin / TPO tab

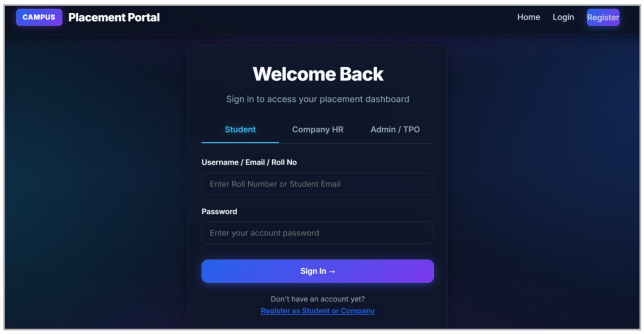


Figure 4.5: Sign-in page, Student tab

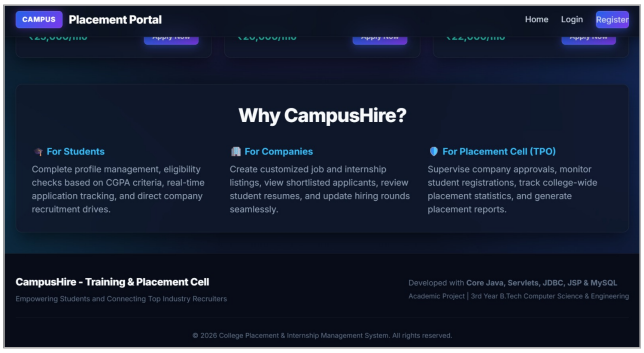


Figure 4.6: Landing page lower section and footer

4. LIMITATIONS AND IMPLEMENTATION REVIEW

4.10 Current Documentation Limitations

- The actual repository, running application and SQL script contents have not been inspected.
- Exact API namespace, dependency versions, route mappings and database fields are unknown.
- Runtime AI functions were confirmed in principle but not described individually.
- Module ownership, test logs and deployment evidence were not supplied.
- The directory listing is partial and should not be treated as an exhaustive file inventory.

4.11 Application Review Checklist

Artifact	Review question
pom.xml	What JDK, dependencies and web packaging are configured?
AuthFilter and mappings	Which routes are protected, and how are roles checked?
Servlets and DAOs	Where are validation and record ownership enforced?
SQL script	What tables, keys and uniqueness constraints exist?
JSP pages	Which described features have actual views and actions?
AI integration	Which tools can the agent call and who approves writes?

4.12 Completion Criteria

A feature should be described as complete only after its interface, server route and persistence behavior are demonstrated together. Negative tests should confirm rejection of unauthorized operations. The team's instruction to check the application before declaring completeness is preserved throughout the report.

The proposed diagrams, schema and appendices are design aids and should be reconciled with the implementation before final academic submission.

5. CONCLUSION AND RECOMMENDATION

5.1 Conclusion

Campus shire addresses the organization of college placement and internship activity through a Java web application. The supplied architecture separates JSP presentation, Servlet request processing, DAO/JDBC access and MySQL persistence. Sessions and AuthFilter are identified as the login-state and role-access mechanisms.

The report replaces the blueprint's anonymous grievance domain with recruitment-specific actors, opportunity ownership, applicant review, status progression and administrative oversight. It also distinguishes the team's stated use of agentic AI from unverified details of its runtime integration.

5.2 Documented Outcomes

- Recorded the supplied Java MVC stack and directory organization.
- Outlined company, recruiter and TPO responsibilities without unconditional completion claims.
- Defined a proposed recruitment data model, lifecycle and access matrix.
- Prepared test cases and evidence requirements rather than invent observed results.
- Acknowledged AI assistance and recommended human oversight.

5.3 Recommendations for Future Work

First inspect the repository and application, reconcile this report with the actual schema, and execute the validation plan. Then consider improvements such as accessible student views, reliable notifications, policy-aware analytics and controlled AI assistance. These are recommendations, not statements that those features already exist.

For final submission, confirm certificate wording with the guide, obtain real screen evidence and complete the team-contribution record.

6. APPENDICES

Appendix A: Proposed System Diagrams

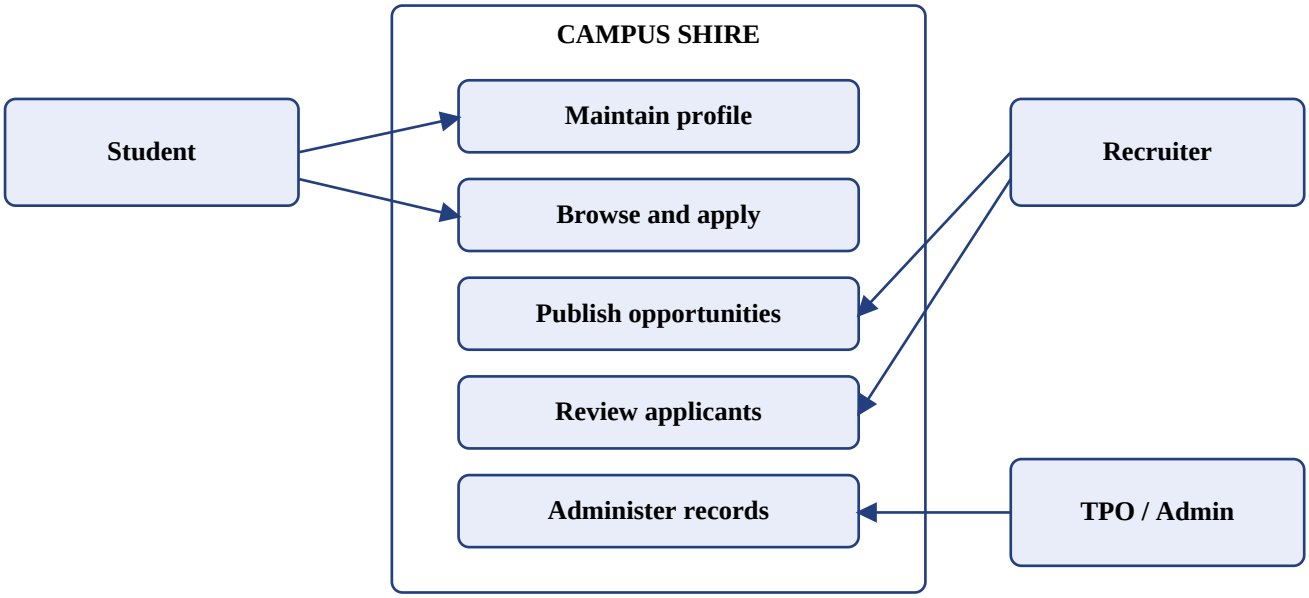


Figure 6.1: Proposed use-case relationships

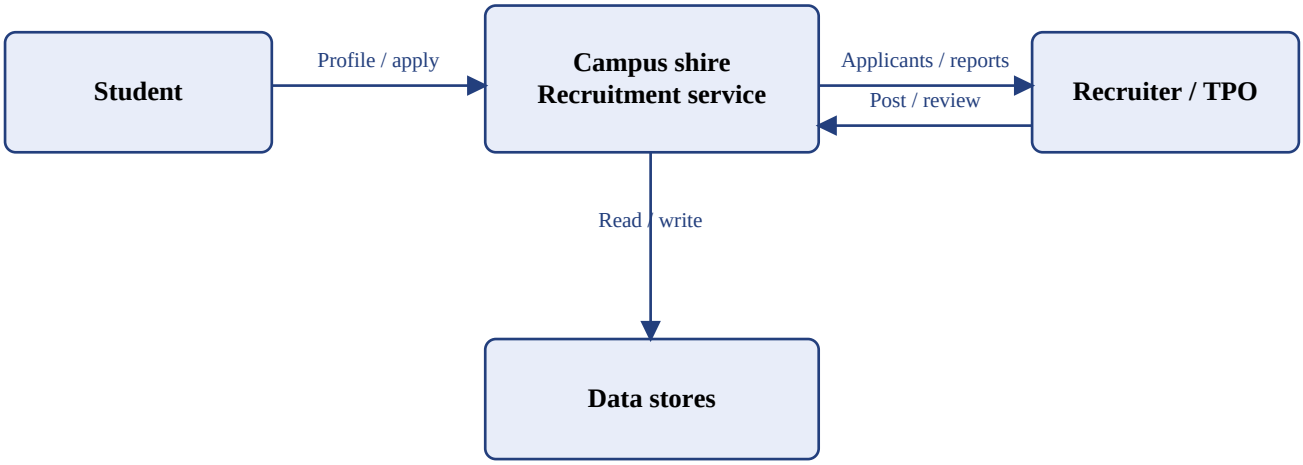


Figure 6.2: Context-level data flow — proposed design

These diagrams explain the recruitment domain. Student functions and reporting scope require confirmation from the current application. They do not represent a reverse-engineered diagram of inspected source code.

6. APPENDICES (CONTINUED)

Appendix B: Application Submission Sequence

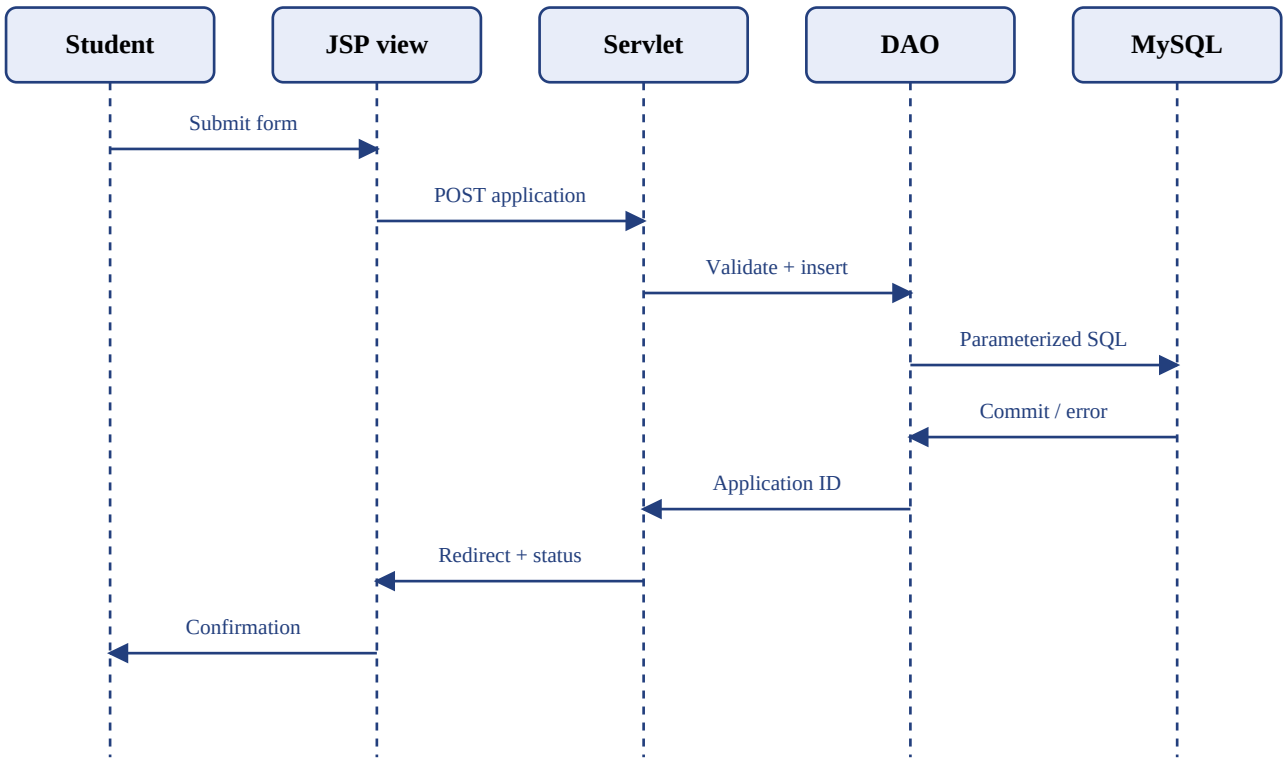


Figure 6.3: Proposed application submission sequence

The sequence begins with a student action and a form submission. The Servlet establishes the actor, validates the request and calls the DAO. The DAO executes a parameterized insert and returns an outcome. The response should distinguish successful persistence from a duplicate or rejected request.

Sequence Preconditions

The student must have a valid authenticated context, the posting must exist and be open under the institution's rules, and the student/posting pair must satisfy any defined eligibility and duplication constraints. These preconditions belong on the server even when a JSP disables a button.

Sequence Alternatives

On invalid input, return a useful error without writing a record. On duplicate submission, preserve one existing application. On persistence failure, roll back the operation and display a controlled failure. These alternatives should be represented in integration tests before the sequence is accepted as implemented.

6. APPENDICES (CONTINUED)

Appendix C: Illustrative Java DAO Example

The following original example demonstrates parameter binding and resource closure for the proposed schema. It has not been extracted from or executed against Campus shire. The example assumes the caller controls the connection and transaction boundary. See reference [2].

```
// Illustrative DAO fragment, not extracted project source.
// Caller must authorize student and posting before this call.
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public final class ApplicationDaoExample {
    public long insert(Connection connection,
                       long studentId, long postingId)
                       throws SQLException {
        String sql = "INSERT INTO applications "
            + "(student_id, posting_id, status) "
            + "VALUES (?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(
                sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setLong(1, studentId);
            ps.setLong(2, postingId);
            ps.setString(3, "APPLIED");
            if (ps.executeUpdate() != 1) {
                throw new SQLException("Insert count mismatch");
            }
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (!keys.next()) {
                    throw new SQLException("Missing generated key");
                }
                return keys.getLong(1);
            }
        }
    }
}
```

Integration Notes

Before integration, map entity names to the real schema, check posting availability and actor ownership, and translate duplicate-constraint errors into a controlled response. Do not trust a student identifier taken directly from the request when the authenticated session already determines the student.

This fragment does not implement authentication, eligibility or a complete application service. A database uniqueness constraint should provide the final duplicate barrier.

6. APPENDICES (CONTINUED)

Appendix D: Illustrative SQL

This SQL is a design example, not the contents of the supplied project's initialization script. Parent-table definitions, key types, deletion policy and storage settings must match the actual database before use. Do not run it over an existing project database without review.

```
-- Illustrative SQL for the proposed schema only.
-- Requires existing students and postings parent tables.
CREATE TABLE applications (
    application_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    student_id BIGINT NOT NULL,
    posting_id BIGINT NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'APPLIED',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_student_posting
        UNIQUE (student_id, posting_id),
    CONSTRAINT fk_application_student
        FOREIGN KEY (student_id)
        REFERENCES students(student_id),
    CONSTRAINT fk_application_posting
        FOREIGN KEY (posting_id)
        REFERENCES postings(posting_id)
);

CREATE INDEX idx_application_posting_status
    ON applications(posting_id, status);
```

Appendix E: Final Evidence Checklist

Required evidence	Current report status
Team names and roll numbers	Supplied and replaced throughout.
Architecture and technologies	Supplied description documented.
Actual features and routes	Application inspection still required.
Screenshots and test results	Not supplied; no fabricated results included.
Runtime AI specification	Specific tool and functions still required.
Guide / HOD signatures	Unsigned; to be completed institutionally.

The project team should use this checklist to turn the report from a documented design draft into an evidence-backed submission. Update only those sections for which the application supplies supporting information.

REFERENCES

[1] Apache Software Foundation. Apache Tomcat Application Developer's Guide.
Available at: https://tomcat.apache.org/tomcat-10.1-doc/appdev/index.html

[2] Oracle. The Java Tutorials: Using Prepared Statements (JDBC).
Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html

[3] OWASP Foundation. Session Management Cheat Sheet.
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[4] OWASP Foundation. Authorization Cheat Sheet.
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[5] Anthropic. Building Effective Agents.
Available at: https://www.anthropic.com/engineering/building-effective-agents

[6] Eclipse Foundation. Jakarta Servlet Specification, Version 6.0.
Available at: https://jakarta.ee/specifications/servlet/6.0/

[7] Eclipse Foundation. Jakarta Server Pages Specification, Version 3.1.
Available at: https://jakarta.ee/specifications/pages/3.1/

[8] Oracle. The Java Tutorials: Lesson – JDBC Basics.
Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

[9] Oracle. The Java Tutorials: Using Transactions (JDBC).
Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html

[10] Oracle. MySQL 8.0 Reference Manual.
Available at: https://dev.mysql.com/doc/refman/8.0/en/

[11] Oracle. MySQL 8.0 Reference Manual: FOREIGN KEY Constraints.
Available at: https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html

[12] Apache Software Foundation. Maven: Introduction to the Build Lifecycle.
Available at: https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html

[13] OWASP Foundation. OWASP Top 10 Web Application Security Risks.
Available at: https://owasp.org/www-project-top-ten/

REFERENCES (CONTINUED)

[14] OWASP Foundation. SQL Injection Prevention Cheat Sheet.
Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[15] OWASP Foundation. Cross-Site Request Forgery Prevention Cheat Sheet.
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[16] OWASP Foundation. Cross Site Scripting Prevention Cheat Sheet.
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[17] OWASP Foundation. Password Storage Cheat Sheet.
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[18] Git Project. Git Documentation.
Available at: https://git-scm.com/docs

[19] Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

[20] Sommerville, I. Software Engineering, 10th ed. Pearson, 2016.

Project-Specific Information

Architecture, technology list, partial directory tree, team names and described company/TPO functions were supplied by the project team in this conversation. Agentic AI use in development and in the application was stated by the team; exact runtime capabilities were not supplied.

The original report was used only as a formatting blueprint. Its student names, grievance algorithm, voting entities, code and performance assertions are not implementation evidence for Campus shire.

References provide technical background, not proof of project completion. No specific Java, Tomcat or MySQL version is asserted for Campus shire.